



JavaScript

講義資料

今回の内容 - 教科書準拠

- 1.1-1 JavaScriptとは？
- 2.1-2 JavaScriptでできること
- 3.1-3 様々なJavaScriptを使ったWebサイト
- 4.2-1 JavaScriptはどこに書く？
- 5.2-2 JavaScriptを書く環境を用意しよう
- 6.2-3 はじめてのJavaScriptを書いてみよう 【実習】
- 7.2-4 JavaScriptを書くときの基本ルール
- 8.2-5 コンソールを使ってみよう 【実習】

最初に知っておこう！ JavaScriptでできること



1-1 JavaScriptとは？

まずは読んでみよう



- Javaとの違いは何だろうか
- 何のために使われるだろうか
- CSSアニメーションとの違いは何だろうか

まずは読んでみよう



- Javaとの違いは何だろうか
 - 名前を参考にしただけで全く異なる！
- 何のために使われるだろうか
 - ページに動的な機能や変化を与える
- CSSアニメーションとの違いは何だろうか
 - リアルタイムやユーザーからの反応など、CSSでは表現できない変化を与える

JavaScriptの活用例

時間軸を与えることができる

JSを使うことによって、時間差や条件に合致した時のみ動作させるような動きが可能になる。きめ細かなアニメーション処理や、より適材適所なデザイン、あるいはカラーリング変更やフォントサイズ変更といったアクセシビリティ対応なども可能になる。

内外とのデータ連携ができる

JSのバックグラウンド処理を併用すると、ページを遷移することなく表示を変更することができる。無駄な描画がなくなり、高速な動作が可能。

例: GoogleMap検索

発展領域が非常に広い

サーバーサイドで動かすnode.js、ウェブからスマホアプリまで作れるReact、PCアプリに強いElectronなど、なんでも作れるとあっていいほど範囲が広い。一昔前までは2大ゲームエンジンの1つにも対応していたほど(廃止)。それを差し引いても、ゲームとマイコン以外はおよそ対応するだけのポテンシャルを持っている。

1-3 様々なJavaScriptを使ったWebサイト

サイト事例を見てみよう

P.18～のサイト事例を見て、今後取り組んでいく内容をチェックしてみよう。

1-x 事前準備

サンプルコードを入手しよう

<https://isbn2.sbcr.jp/15758/>

↓

<https://www.sbcr.jp/support/4815617728/>

→ 「【ダウンロード】『1冊ですべて身につくJavaScript入門講座』 サンプルデータ」：「JS_Data-2.zip」

【保存場所(候補)】

「~/school/cr3/html-css-js/」



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

JavaScriptに触れてみよう！



2-1 JavaScriptはどこに書く？

記述場所のルール

- HTMLファイル内に直接書く方法
 - headやbody内の任意の場所に記述できる
 - HTML内に書く場合は「<script></script>」で囲ったタグの中に書く必要がある
 - 共通処理があった場合は対象の全てのファイルに同じ内容を書く必要がある
- 外部ファイルとして書きHTMLから読み込む方法
 - 拡張子は「.js」とし、HTMLと違い<script>タグは省略する
 - HTMLから<script>タグで読み込ませる

2つの扱い方のまとめ

- <script>タグはHTMLに直接jsを書く方法と、外部jsファイルを読み込む方法の両方に使われる
- jsは記述方法にかかわらず、複数配置することができる

2-2 JavaScriptを書く環境を用意しよう

開発環境について

- エディタ: Visual Studio Code
- プラグイン
 - 日本語化プラグイン(任意)
 - Live Server
- データ管理: Git, SourceTree

2-3 はじめてのJavaScriptを書いてみよう【実習】

コード実習の進め方について

- ✓ 自分のプロジェクトを作って実際に書いてみる
- ✓ コードはサンプルコピペせず「自分で」書くこと(单元によって変える可能性あり)

【保存場所(候補)】

「~/school/cr3/html-css-js/」, ディレクトリ名: 「JS_MyProject/chapter2/demo○○」
デモファイルと同じ場所で構わない(日本語は使わない)。特に後期の内容は日本語データが動かないリスクが高いため注意。

名前	更新日時	種類	サイズ
JS_Data-2	4/16/2025 9:48 AM	ファイル フォルダー	
JS_MyProject	4/16/2025 9:51 AM	ファイル フォルダー	

内部と外部の記述方法

HTML内部に書く方法

【インラインスクリプト】

HTML文書内にスクリプトを書く行為を「インラインスクリプト」などと呼ぶ。『インラインで書いて』と言われたらこの書き方。

✓ 通常のHTMLでは非効率な場合が多いが、CMSやフロントエンドでは活躍する

✓ 記述順序が煩雑になるリスクがあるので乱用には注意すること

専用のjsとして外部ファイル化して書く方法

【外部スクリプト】

外部のスクリプトを「src」属性で読み込むものを「外部スクリプト」などと呼ぶ。jQueryをはじめとしたライブラリを使う場合は全てこちらにあたる。

✓ 同期、非同期によって表示速度やスコアに影響する可能性がある(いずれ実習する)

✓ ファイルにまとまっているため、流れを掴みやすくバグの原因が特定しやすい

2-3 はじめてのJavaScriptを書いてみよう【実習】

実習を試みよう

2-3デモのコードを実際を書いてみよう

- 教科書に書いてあるコードを記述しLiveServerを起動する
 - 動作が書いてある内容とあっているかを確認する
 - ダウンロードしたサンプルコードに答えはあるが、まずは自分で書いてみて躓きポイントがどこにあるかを理解すると良い
 - インテリセンス機能は積極的に活用する
- 今回の実習では2つの記述方法を試すが、必ず順番通りに実行すること(迷い防止)
- 終わった、分からないときは伝えること



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

文法や用語を確認しよう

【ベースとなる用語について】

- 「オブジェクト」：JavaScriptで汎用的に表すデータのかたまりで、数値型なども含まれる
- 「メソッド」：特定の動作をする処理のかたまり。例) 警告文を出す、文章をログに表示する
- 「パラメーター」：よく使われる意味のままで、上記のメソッドの例で言えばメソッドに渡す内容に応じて警告文やログの内容が変わる ※教科書ではあまり使われない「パラメーター」表記

【初歩の記述ルールについて】

- 1文の終わりにセミコロン「;」を記述する(例外あり)
 - 「;」を自動で付与する仕様があるが、少なくとも最初は自分で付けたほうが安心
- 「命令文」は半角入力を使う(全角ではエラーになる)
- 大文字と小文字は区別する(命名ルールに着目しよう)
- 文字列には引用符を使い、また日本語全角なども使用可能
 - 外国サービスなど稀に例外があるため臨機応変に

✓教科書の「2-4」を読んで詳細を理解しよう。



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

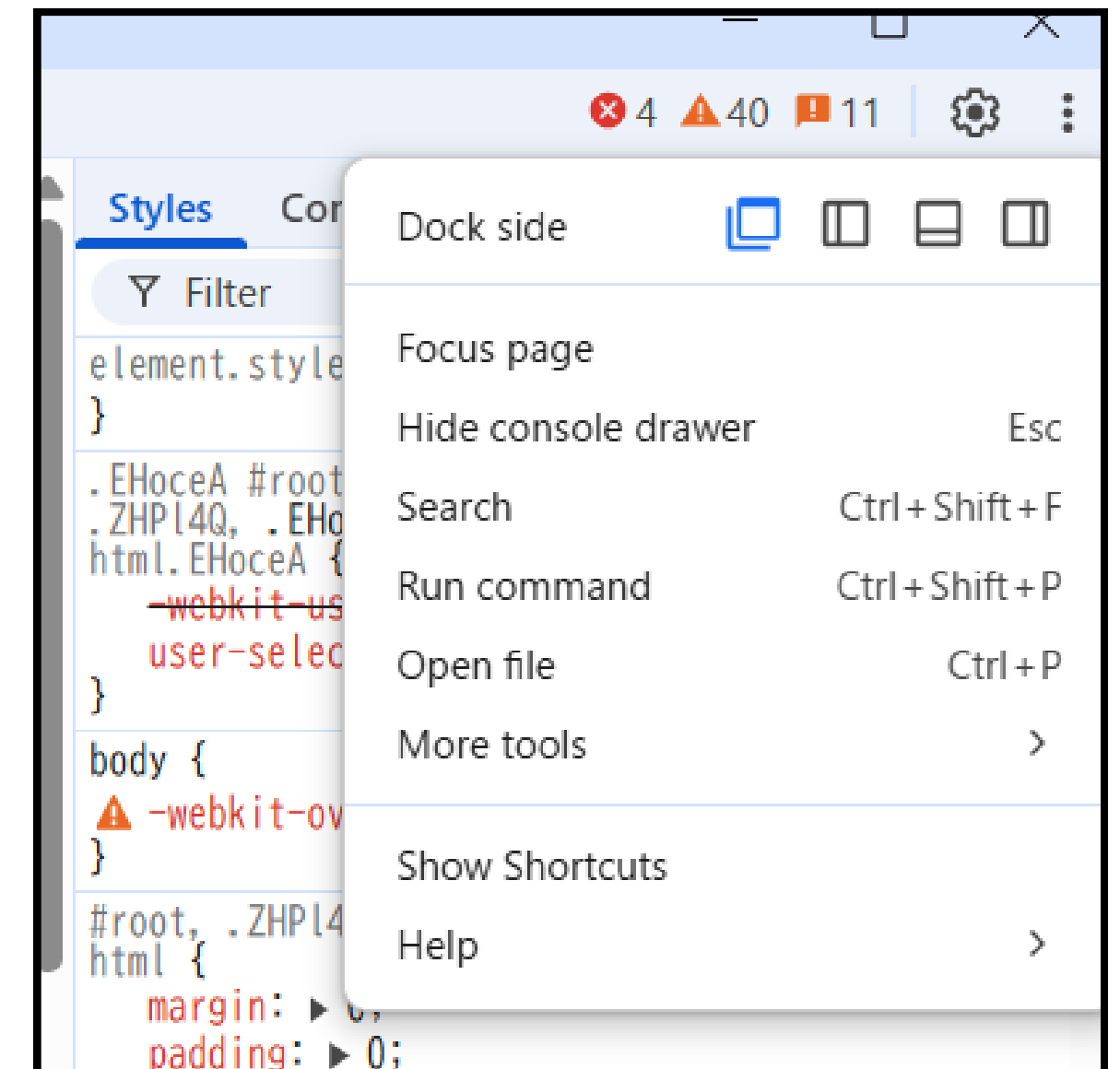
2-5 コンソールを使ってみよう【実習】

開発者ツールを使ってみよう

開発者ツール

JavaScriptのデバッグ(動作確認)の他、開発者に対して警告やメッセージなどを伝えるためのツール。ブラウザから対象のページ上で、「右クリック」または「F12」ショートカットで起動する。設定によって別ウィンドウ化(右図)もできる。

✅教科書の「2-5」を読んで、コンソールに文字を表示させ確認する方法を学ぼう



開発者ツールを使ってみよう

デバッグの方法

JavaScriptに限らずプログラミング言語ではエラーが出る、思っていた動作と異なる、など思い通りにいかないことは頻繁に発生する。

開発者ツールを使うとJavaScriptでデバッグができるため、今のうちに方法を確認しよう。

デバッグを行いたい箇所にマークを付けておくと、その処理が実行される直前にプログラムが一時停止する。

✅ 「開発者ツール」を起動→「Source」タブ→ターゲットの「jsファイル」を表示→処理を停止したい行の左側をクリック(※宣言だけなどの具体的な処理をしない位置には設定できない)

✅ コンソールの命令でマークを入れた時、どのように動作したか確認しよう



ここまでで
質問はありますか？

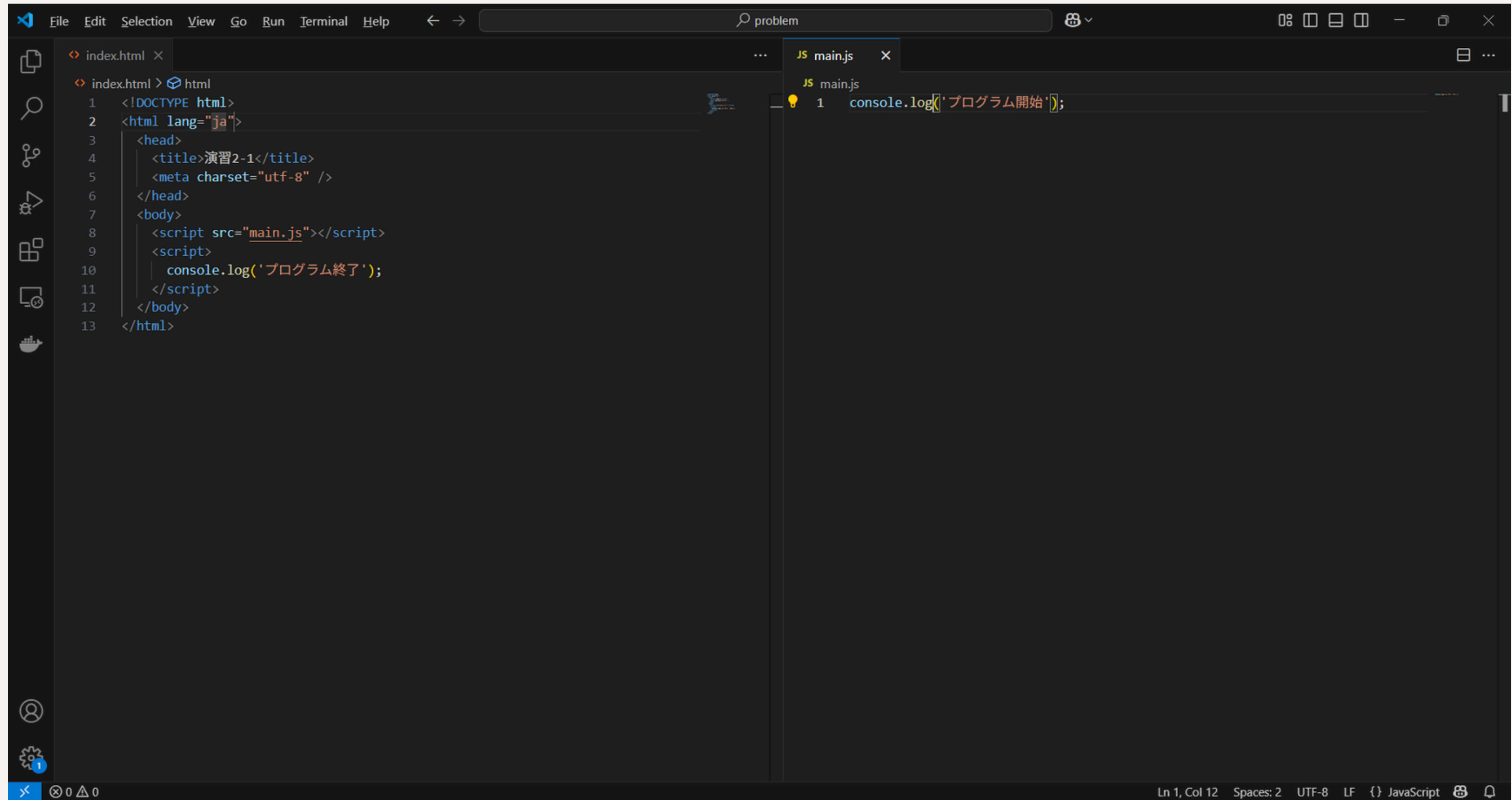
遠慮せずに挙手・発言してください！

演習2-1

以下の条件に合致するコードを記述してすること。

1. 「JS_MyProject/chapter2/problem」ディレクトリを作成する
2. 上記に「index.html」と「main.js」を作成する
3. 「index.html」に、**コンソール**で「プログラム終了」と表示するプログラムを作成する
4. 「main.js」では、**コンソール**で「プログラム開始」と表示するプログラムを作成する
5. 「プログラム開始」「プログラム終了」という順で表示されるようにすること
6. タイトルなどは任意

演習2-1 - 回答例



```
index.html > html
1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <title>演習2-1</title>
5     <meta charset="utf-8" />
6   </head>
7   <body>
8     <script src="main.js"></script>
9     <script>
10      console.log('プログラム終了');
11    </script>
12  </body>
13 </html>
```

```
JS main.js
1 console.log('プログラム開始');
```

Ln 1, Col 12 Spaces: 2 UTF-8 LF {} JavaScript

今回のまとめ



2つの記述方法

文書内に直接書く方法と、外部ファイルに分ける方法を学んだ。将来的にはプロジェクトに応じて使い分けるようにする。



簡単なメソッドを使用

alertやconsoleなどのオブジェクトに属するメソッドを使用した。今後も使っていくため、使用方法は覚えておくこと。



JavaScript

実習2 - ページの書き換え／定数／条件分岐

今回の内容 - 教科書準拠

- 1.3-1 作成するカラーピッカーの紹介
- 2.3-2 必要なファイルを用意しよう【実習】
- 3.3-3 カラーピッカーの色の値を取得しよう【実習】
- 4.3-4 テキストを変更しよう【実習】
- 5.3-5 DOMを理解しよう
- 6.3-6 テンプレート文字列で表示させよう【実習】
- 7.3-7 定数でコードをスッキリまとめよう【実習】
- 8.3-8 カラーコードを表示する「きっかけ」を作ろう【実習】
- 9.3-9 関数で選んだ色を取得しよう【実習】
- 10.3-10 ページの背景色を変えてみよう【実習】
- 11.3-11 条件を付けて色の名前を表示させよう【実習】

JavaScriptの基本を学ぼう！



Chapter3のポイント

1. JavaScriptで要素(タグ)を取得する
2. 要素のプロパティを取得・更新する
3. 外部ファイルの遅延読み込みを理解する
4. DOM構造について理解する
5. コメントアウトとその方法について理解する
6. 便利な「テンプレート文字列」を使った記法を学ぶ
7. 定数を理解する
8. イベントの触りを理解する
9. 関数の定義を理解する
10. 条件分岐を理解する

3-2 必要なファイルを用意しよう【実習】

deferについて

- defer: スクリプトの非同期読み込みで、deferで宣言した順に実行される
- async: スクリプトの非同期読み込みで、順不同で実行される

defer

```
<script src="jquery.js" defer></script> 順1  
<script src="jquery.ext.js" defer></script> 順2
```

defer同士の順序が守られるため基本はこちらを使いたい。deferとasync、deferと未記載(同期読み込み)の混在では順序が崩れるため、依存するファイル同士は同じルールで記述しよう。

async

```
<script src="jquery.js" async></script>  
<script src="jquery.ext.js" async></script> 順?  
順?
```

読み込み順が不明なため、完全に独立したファイルでのみ使うことができる。いずれにせよ最終的には読み込むため、最も早いというものでもない。

3-3 カラーピッカーの色の値を取得しよう【実習】

コメントアウトについて

- 「//」を記述した行はコメントとして処理されJSが実行されない
- 「/*」と「*/」で囲んだ範囲は全てコメントとして処理される

複数行コメントの注意点

```
1  /*
2  <script>
3  /*console.log('コメントアウト1');
4  console.log('コメントアウト2');*/
5  </script>
6  */
```

複数行コメントで囲う場合、二つ以上のコメントアウトが干渉する可能性がある。
良く起こることではないが、まとめて作業する時など見逃さないように注意すること。

コメントの鉄則(リーダブルコードより)

- コードから読み取れることを書かない、価値の提供できないコメントは書かない
- コメントを書くなればコメントのメンテナンスもセットで行う
- 変数の名前、条件の指定など、プログラムのロジックで解決できることをコメントで済まそうとしない

DOMの活用方法

DOM: HTMLの構造を表した組織図のようなもので、その形からDOMツリーなどとも呼ばれる。

※家系図: ファミリーツリーと同様に構造を表すということ。

どのようなところで役立つのか？

CSSのセレクターやJavaScriptの要素指定で、親子・兄弟関係を使うことがよくある。

特に、複数同じ項目がある場合は「どの」「何」なのかを指定しなくてはならない。

例)

ログインフォームのメールアドレス情報と、お問い合わせフォームのメールアドレス情報

3-6 テンプレート文字列で表示させよう【実習】

文字列の連結方法

文字列同士をそのまま繋げる場合

数値のように足し算でつなげることが可能。

例) `console.log( '私の名前は' + name + 'です。' + age + '歳です' );`

テンプレート構文でスッキリまとめた場合

バッククォート「```」で全体を囲い、変数や関数を使う場所では「`${}`」の中括弧内に記述する。

例) `console.log(`私の名前は${name}です。${age}歳です`);`

3-7 定数でコードをスッキリまとめよう【実習】

定数constの細かいルール

- 数値や文字列などを指定した場合は再代入ができない
- オブジェクトや配列を指定した場合は、プロパティや中のデータ編集は可能
 - あくまで「参照先」を保存しているだけ

例)

top

under



```
const books = under;  
under.append( 'js' : 'javascript入門講座' );
```

あくまで本棚の一番下の場所を記憶したような処理になる。したがって、本を取り出したり、入れたりといったことは可能。

ただし、「books」を改めて一番上「top」の段に設定し直すなどは不可。

3-8 カラーコードを表示する「きっかけ」を作ろう【実習】

イベントとは？

イベントとは？

きっかけや起こりという捉え方。AをしたらBをするといったもの。

Webサイトでは様々あるが一例を掲載する。

- ページの読み込みが終わったら
- ある要素がクリックされたら
- windowサイズが変更されたら
- フォームの内容が変更されたら

それぞれ必要に応じて「〇〇されたら」の「〇〇」と、その後の処理内容をプログラムとして記述する。

3-9 関数で選んだ色を取得しよう【実習】

関数とは？

関数とは？

一定の機能を持たせて処理をまとめたもの。

登場したものでは「alert」や「console.log」などが該当する。

効率化や見通し、再利用性などのためにはまず必須となる考え方。

逆に言うと、効率などを一切考えなければ自分で関数を用意する必要はない。

✅ バグをなくす、原因を早く特定するといった目的にはまず必要なため必ず覚えよう。

記述方法はいくつかあるが、紹介されているアロー関数が現在ではよく使われる。

✅ よくわからない記述があれば、アロー関数の「短縮形」を疑ってみよう

✅ アロー関数と宣言型の関数では仕様上の違いもある(複雑なので割愛するがthisの取り扱いなど)

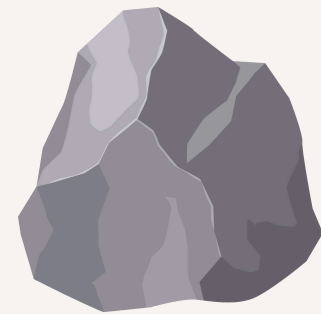
3-11 条件を付けて色の名前を表示させよう【実習】

条件分岐の考え方

- 日常生活に置き換えて考えてみよう



「もしも」障害物があったら



「もしも」プリンに名前が
書いていなかったら



3-11 条件を付けて色の名前を表示させよう【実習】

条件分岐と「型」

- 「==」と「===」の違いとは？
 - 「==」では型を考慮しない(暗黙的に型変換を行う)
 - 「===」では型を考慮する
- 「<=」や「>」などの不等号でも型の変換が行われる

===

- 両方とも一致しない

```
if (1 === '1' )
```

```
if (0 === null)
```

==

- 両方とも一致する

```
if (1 == '1' )
```

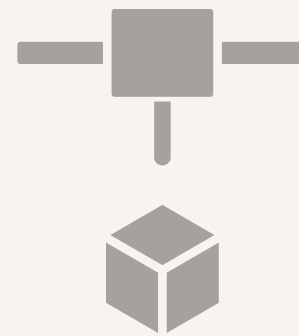
```
if (0 == null)
```

今回のまとめ



関数

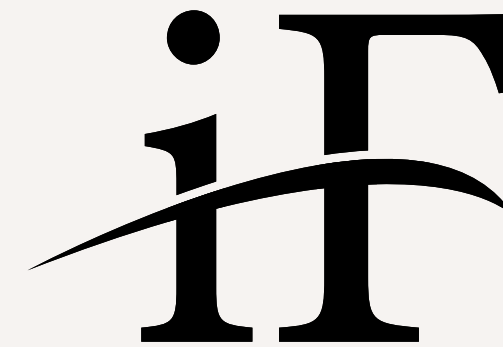
いつまでも使うであろう関数の基本を学ぶ。アロー関数と通常の関数の違いを理解する。



定数

定数の宣言方法と例外的なルールを学ぶ。大きなプロジェクトでもバグの少ないプロジェクトの構成を理解する

。



条件分岐

基本的な条件分岐の使い方を理解する。ほんの少しの違いでバグの原因にもなる「型」違いを理解する。



JavaScript

実習3 - イベントで操作しよう！ - ロードイベント／クリックイベント

今回の内容 - 教科書準拠

- 1.4-1 イベントとは？
- 2.4-2 ローディング中の画面を作ろう【実習】
- 3.4-3 CSSのクラスを加えよう【実習】
- 4.4-4 ボタンをクリックしてダークモードにしよう【実習】
- 5.4-5 CSSのクラスを切り替えよう【実習】

イベントで操作しよう！

E

Chapter4のポイント

1. イベントについて理解を深める
2. クラスとイベントの組み合わせ方法を学ぶ
3. イベントで状態(クラス)を管理する
4. イベント×条件分岐の使い方を理解する
5. 複数回発生するイベントを使えるようになる
6. TIPS. 条件分岐の複数条件とよくあるミスを理解する

4-1 イベントとは？

イベントの基本

イベントの基本

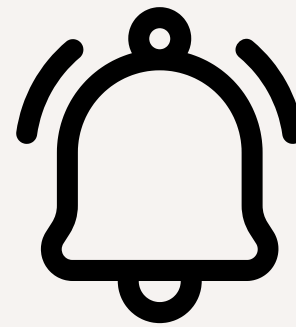
HTML／CSSなどで作ったページにJSで時間軸を与えることができる。

そして、具体的な行動(アクション)があったら特定の行動をとるための仕組みを「イベント」という。



スマホを鳴らす

電話がきたら...



講義が終わる

チャイムが鳴ったら...

4-3 CSSのクラスを加えよう【実習】

windowやdocument

- 「window」や「document」など、どのイベントに設定するかはリファレンスを確認する
 - <https://developer.mozilla.org/en-US/docs/Web/API/Window>
 - <https://developer.mozilla.org/en-US/docs/Web/API/Document>

習うより慣れよ

今日の講義のまとめ

E

イベント

イベントの基本を学び、この後学んでいく条件分岐やデータとの連携を見据えた下地を作る。



JavaScript

実習4 - イベントで操作しよう！ - 条件分岐と組み合わせ／スクロールイベント

今回の内容 - 教科書準拠

- 1.4-6 if / else文で表示する文字を切り替えよう
- 2.4-7 入力した文字数を数えてみよう【実習】
- 3.4-8 lengthでカウントしよう【実習】
- 4.4-9 文字数によって表示を変えよう【実習】
- 5.4-10 チェックをいれるとボタンを押せるようにしよう【実習】 
- 6.4-11 チェックでボタンを有効化しよう【実習】
- 7.4-12 より効率のいい書き方を考えよう【実習】
- 8.4-13 ページのスクロール量を表示しよう【実習】
- 9.4-14 スクロール量を取得しよう【実習】
- 10.4-15 ページのサイズを取得しよう【実習】
- 11.4-16 計算式を書いてみよう【実習】

4-8 lengthでカウントしよう【実習】

lengthとは？

JavaScriptの標準オブジェクト

JavaScriptでは、様々なものが「オブジェクト」として定義されています。

イベントで使用している「document」や「window」だけでなく、数値や文字列も実はオブジェクトです。
したがって、同様に標準機能として「length」をはじめとしたプロパティやメソッドが提供されています。

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/String

4-12 より効率のいい書き方を考えよう【実習】

論理式の応用方法

論理式の計算結果は真理値である

※真理値: 「true」 / 「false」

if文の中で使われる真理値は、計算結果が真理値になります。
チェック状態などの変数だけでなく、関数の判定などにも使われます。

例)

```
function isPositiveValue(num) {  
    return num >= 0; // 0以上ならtrue, 0未満ならfalse  
}
```


4-16 計算式を書いてみよう【実習】

論理演算子とビット演算

if文でのビット演算ミスに注意

JavaScriptを始め多くのプログラミング言語でビット演算が使用できます。

よく似た式なので、記述ミスに注意しましょう。

今回は「AND」を取り上げていますが、「OR」や「XOR」(ほぼ使わない)なども存在します。

【論理式】

```
if (a === b && a === c)
```

【ビット演算(エラーにならず式としても成り立つ)】

```
if (a === b & a === c)
```

ビット演算「&」の詳細)

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Operators/Bitwise_AND

今日の講義のまとめ

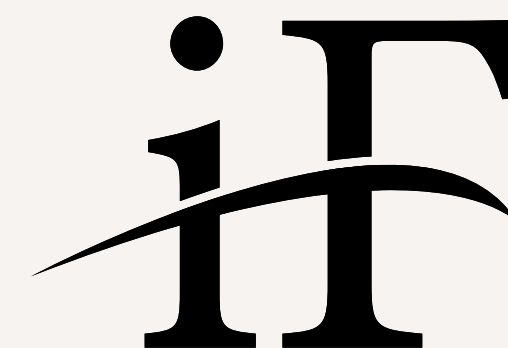


イベント

イベントの基本を学び、この後学んでいく条件分岐やデータとの連携を見据えた下地を作る。



組み合わせ



条件分岐

基本的な条件分岐の使い方を理解する。ほんの少しの違いでバグの原因にもなる「型」違いを理解する。



JavaScript

実習5 - 複数のデータを使ってみよう！ - 繰り返し文

今回の内容 - 教科書準拠

1.5-1 作成する画像一覧ページの紹介

2.5-2 InsertAdjacentHTMLでHTMLタグを挿入しよう【実習】 

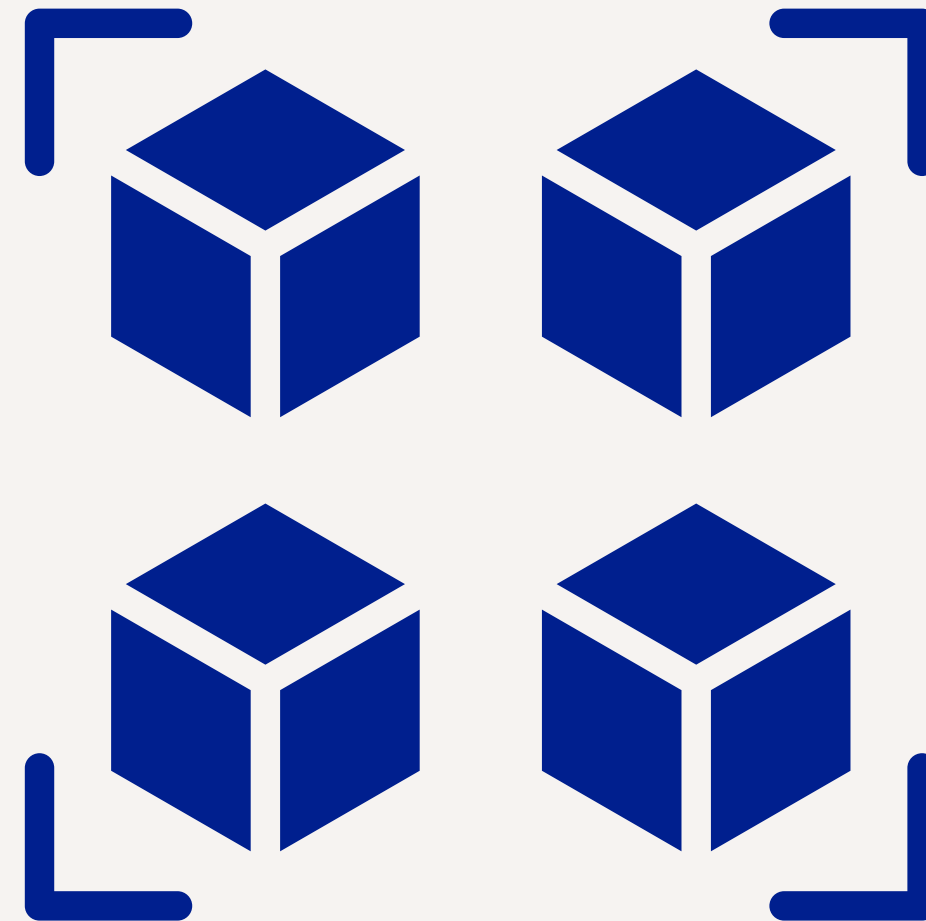
3.5-3 配列で複数の画像のファイル名をまとめてみよう【実習】

4.5-4 配列の中にある画像を表示しよう【実習】

5.5-5 for分の繰り返し処理を理解しよう【実習】 

6.5-6 for分で画像を一覧表示しよう【実習】

複数のデータを使ってみよう



Chapter5のポイント

1. JavaScriptからHTMLを追加する方法を学ぶ
2. 配列の使い方を理解する
3. 繰り返し文(ループ処理)を理解する
4. 配列、オブジェクト、ループの組み合わせを理解する

概要と目的

概要

- JavaScriptで表示する画像や情報を記述する

目的

- あらかじめ設定した情報を元にページを生成できるようになる
- 動的に情報を変更する、ということを理解する - SNSにならって



個別にHTMLを一々書かなくとも、SNSなら投稿、商品なら商品登録で、決められたフォーマットで情報を生成できる。

X/Youtube/Instagram/Amazon/楽天/その他

変数とWebフォント

変数とIDの制約について

変数やIDとして使えない場合(予約済みの記号など)を除いて、大きな制約はありません。

【○同じIDを複数の変数で指定する】

```
const menu1 = document.querySelector( '#menu' );  
const menu2 = document.querySelector( '#menu' );
```

Webフォントをcssから読み込む方法

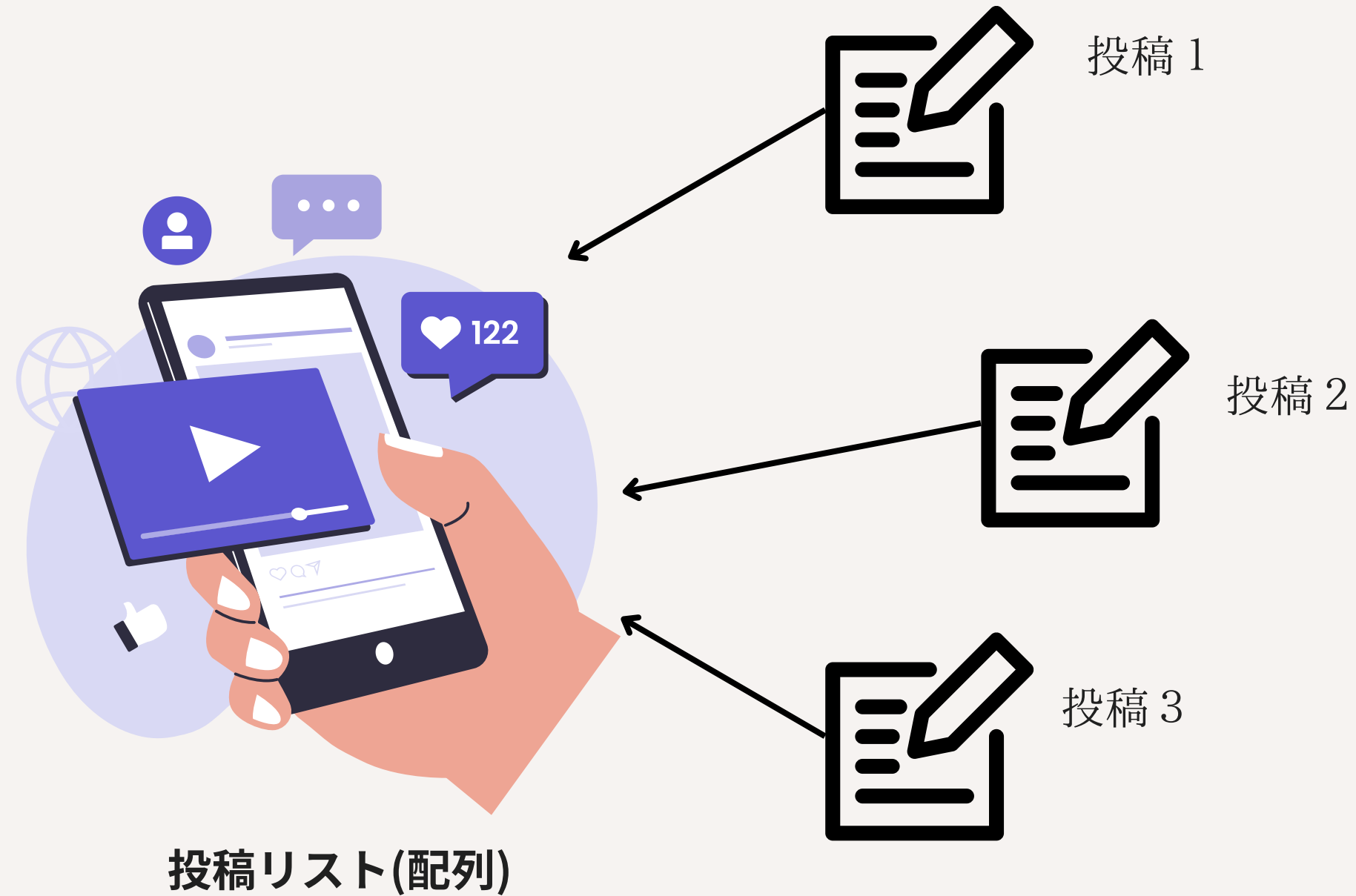
- @font-faceを使うと、CSSから読み込むことも可能です。
- <https://developer.mozilla.org/ja/docs/Web/CSS/@font-face>

5-3 配列で複数の画像のファイル名をまとめてみよう

配列の考え方

SNSで考えてみよう

配列×オブジェクト(5-8)



5-5 for分の繰り返し処理を理解しよう

インクリメント処理

インクリメントの意味

for (let i = 0; i < 10; **i++**) ← 「**i++**」：インクリメント、変数の値を『1ずつ増加』させる時に使う

■以下のコードは意味として同じ

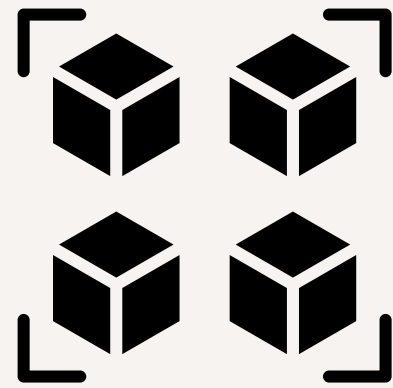
- `i = i + 1; // 標準形`
- `i += 1; // 上記の短縮記法(加算代入)`
- `i++;` // `i`を『1ずつ増分する』という状況でのみ使用できる構文、主にループ用
- **×** `i++++;` // `i`を2ずつ増やすときは、標準形か短縮記法を使う

■デクリメント処理

インクリメントとは逆に、『変数の値を1ずつ減少』させる時に使う

- `i--;` // その他の表記を含めて、加算の記号「+」を「-」に変えるだけ

今回のまとめ



配列

データを一元的に、効率よく管理する方法を学ぶ(活用方法は次回以降)



繰り返し

繰り返しはプログラム化(自動化)の第一歩。しっかり使いこなせるようになろう。



JavaScript

実習 6 - 複数のデータを使ってみよう！ - 変数の宣言と記法について

今回の内容 - 教科書準拠

- 1.5-7 変数letと定数constの違いとは？
- 2.5-8 オブジェクトで画像、メニュー名、値段をまとめよう
- 3.5-9 オブジェクトの情報を取得しよう

5-7 変数letと定数constの違いとは？

letとconstの使い分け

constのルールと使い分けについて

- 再代入が必要な場合: let
- 一度しか代入を行わない場合: const
- オブジェクトや配列の参照先が変わらない場合: const



- 固定値の場合…左図の特定の本を示すイメージ。書き換えることはできない。
- 参照値の場合(配列、オブジェクト等)…左図の本棚を示すイメージ。棚の本を入れ替えたり、特定の本を上書きすることができる。別の本棚を指定することができない。

```
const books = document.getElementById( 'books' );  
books.innerText = '新刊' ; // constでも可能
```

5-8 オブジェクトで画像、メニュー名、値段をまとめよう

オブジェクトの捉え方

表計算ソフトの行列の関係をイメージしよう

名前	価格	数量
りんご	450	1

【書式】
{
 名前: ‘りんご’ ,
 価格: 450,
 数量: 1
}

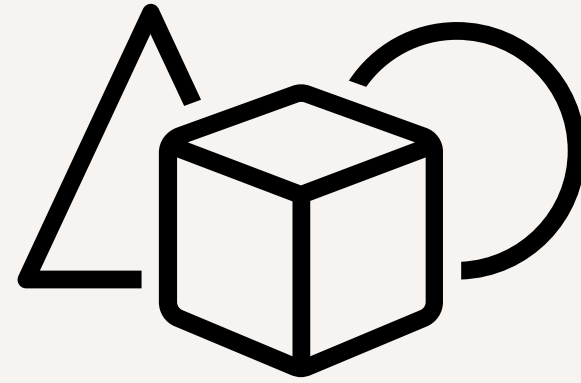
配列を組み合わせると、行が追加されるイメージでリンゴ以外のデータも一覧として管理できる。

ドット記法の利点

TypeScript/設定次第でドット記法のほうが有利

- 変数名+ドットまで打つと、インテリセンス機能を利用できる
 - 指定が間違っているとundefinedになる点は同じ
- ループを組み合わせる場合は「ブラケット」記法を使うパターンがある(5-9参照)

今回のまとめ



オブジェクト

JSの最も使うデータと言っても過言ではない。データを一つだけ使うというのはむしろ稀なため、使い方は覚えておきたい。



JavaScript

実習7 - 複数のデータを使ってみよう！ - 画像一覧を完成させよう

今回の内容 - 教科書準拠

- 1.5-10 配列とオブジェクトを組み合わせでデータをひとまとめにしよう
- 2.5-11 一覧表示をしよう
- 3.5-12 分割代入でコードをスッキリさせよう



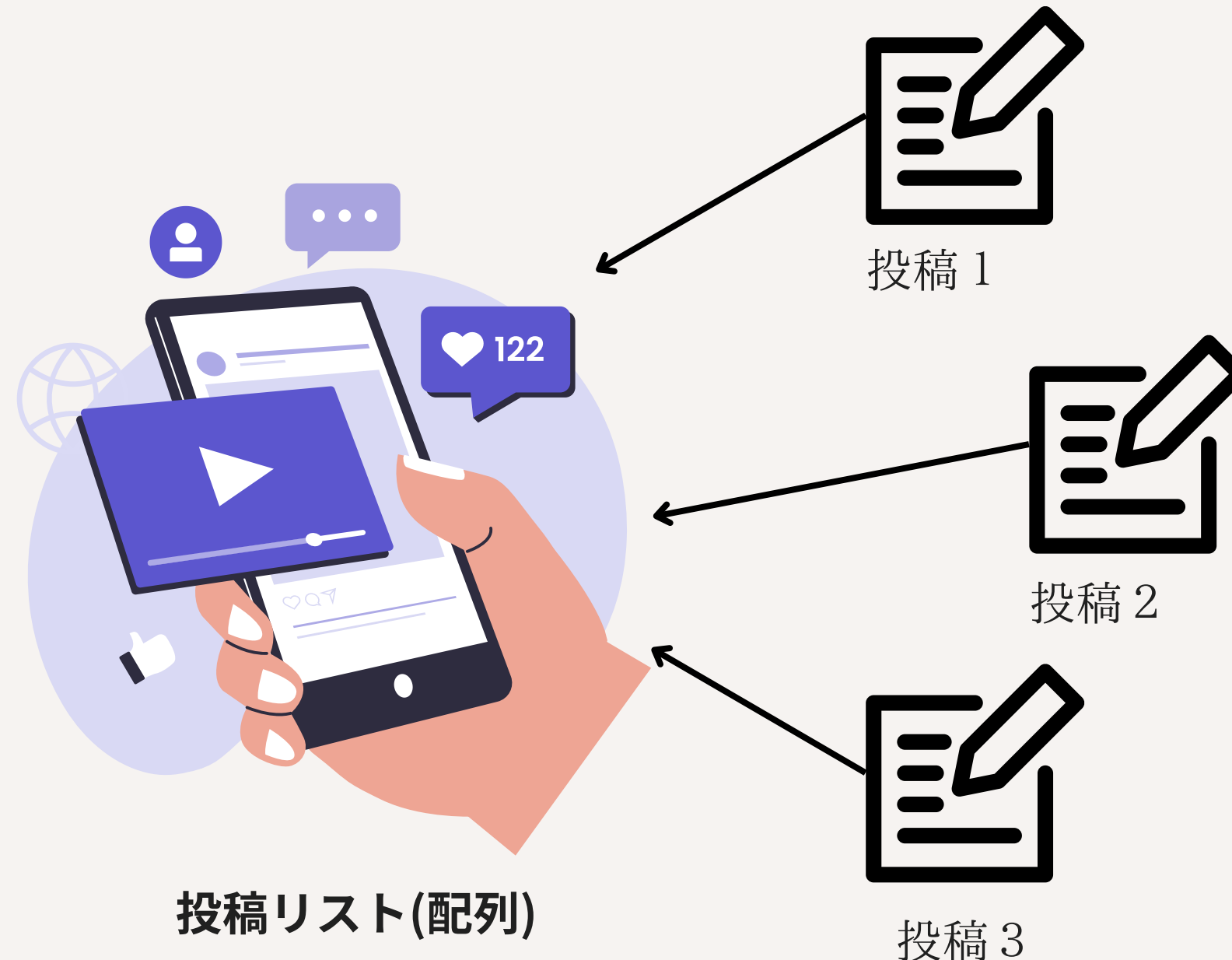
ここで足並みを揃える

5-10 配列とオブジェクトを組み合わせデータをひとまとめにしよう

配列による宣言

【再掲】SNSで考えてみよう

配列×オブジェクト



それぞれの投稿に日時や編集情報、投稿者情報、そして内容が紐づく。

各情報がオブジェクトのキー・バリューに該当し、投稿一つ一つを一覧として管理したものが配列に該当する。

5-11 一覧表示をしよう

教科書を進めよう

以下のポイントに注意しよう

- テンプレート文字列内ではHTMLのインテリセンスが効かない
- 開始タグや閉じタグの記述ミスをするとうまく表示されない
- JSでタグの属性を追加するときはクォートも必要になり、コピペも含めて抜け落ちしないようにすること

5-12 分割代入でコードをスッキリさせよう

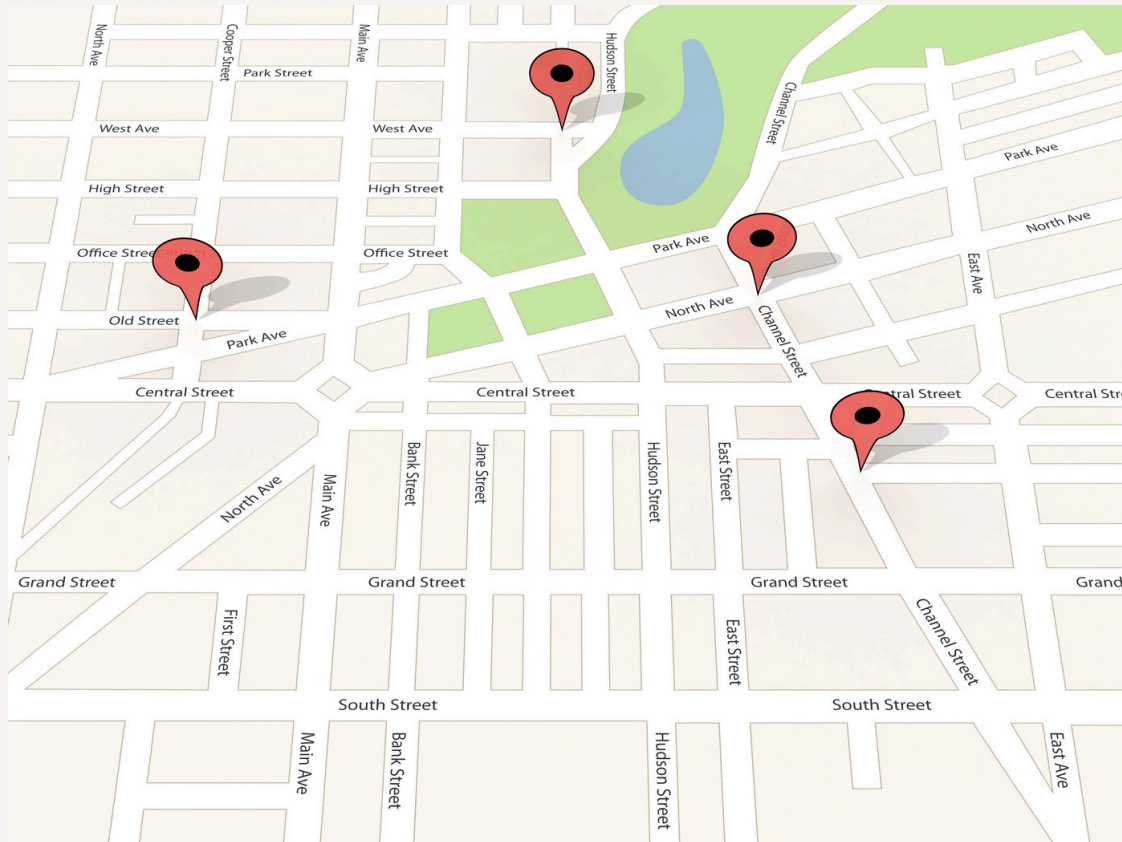
配列とオブジェクト

オブジェクト単体、またはオブジェクトを配列にして使われる

JavaScriptではjQueryを始め、通常ライブラリを組み合わせで記述する。

エンジニアが行うことは、順番や条件を指定すること。

また、外部API(例: GoogleMap)などを使うと、配列×オブジェクトのデータが当たり前のように取得される。



例) 配列として、ピン一つ一つの情報取得される



例) オブジェクトとして、それぞれのピンに設定された緯度・経度、建物情報、住所、その他様々なデータ取得される

文法の基本はここまで！

文法の概要

- JavaScriptの主な文法は5章までで完結
- 標準関数やライブラリを組み合わせで発展させる
- ES6という規格で追加された構文で更に便利な使い方ができる(アロー関数など一部使用済みのものも存在する)
- この機会に振り返ってみよう

文法を簡単に振り返ろう

「何」には用語、「○」には該当する文字(漢字含む)を入れてみよう

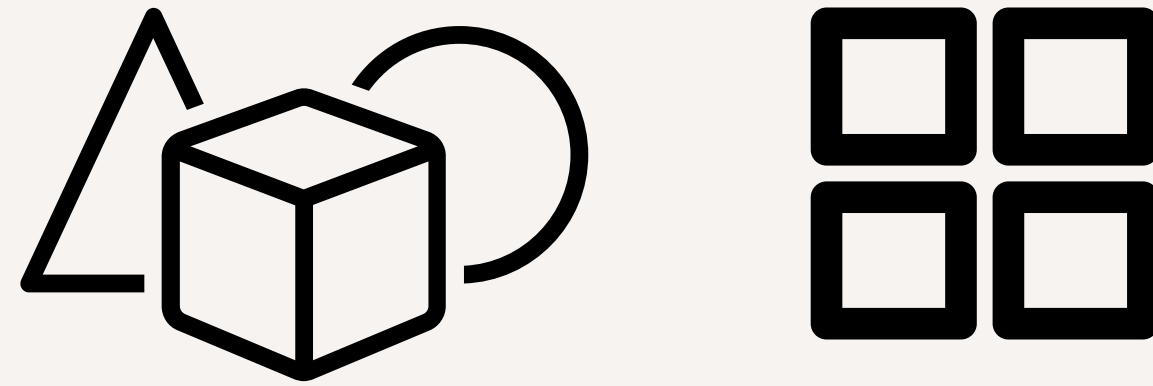
- HTMLにJSを書く/読み込むときは…「何」タグ
- querySelectorやgetElementByIdは…「何」オブジェクト
- 条件分岐は…もし: ○○/それ以外: ○○○○/それ以外でもし: ○○○○ ○○
- 何々があったら何々をする(トリガー): ○○○○, 例) click○○○○, window○○○○
- ループは…○○○命令
- key: valueでデータを管理するものは…○○○○○○○
- 一覧(リスト)でデータを管理するものは…○○
- 後で値を書き換える変数は「何」、書き換えない定数は「何」で宣言する
 - 「宣言」 name = 'My Name';
 - 「宣言」 obj = document.querySelector('#object');

文法を簡単に振り返ろう

「何」には用語、「○」には該当する文字(漢字含む)を入れてみよう

- HTMLにJSを書く/読み込むときは…「**script**」タグ
- querySelectorやgetElementByIdは…「**document**」 **t**オブジェクト
- 条件分岐は…もし:「**if**」/それ以外:「**else**」/それ以外でもし:「**else if**」
- 何々があったら何々をする(トリガー):「**イベント**」, 例)「**clickイベント**」, 「**windowイベント**」
- ループは…「**for**」命令
- key: valueでデータを管理するものは…「**オブジェクト**」
- 一覧(リスト)でデータを管理するものは…「**配列**」
- 後で値を書き換える変数は「**let**」、書き換えない定数は「**const**」で宣言する
 - **let** name = '**My Name**';
 - **const** obj = **document**.querySelector('#**object**');

今回のまとめ



オブジェクト×配列

「JavaScriptでデータを扱う」とは配列とオブジェクトの使えるようになること。ループや条件分岐と組み合わせられるようになるう。



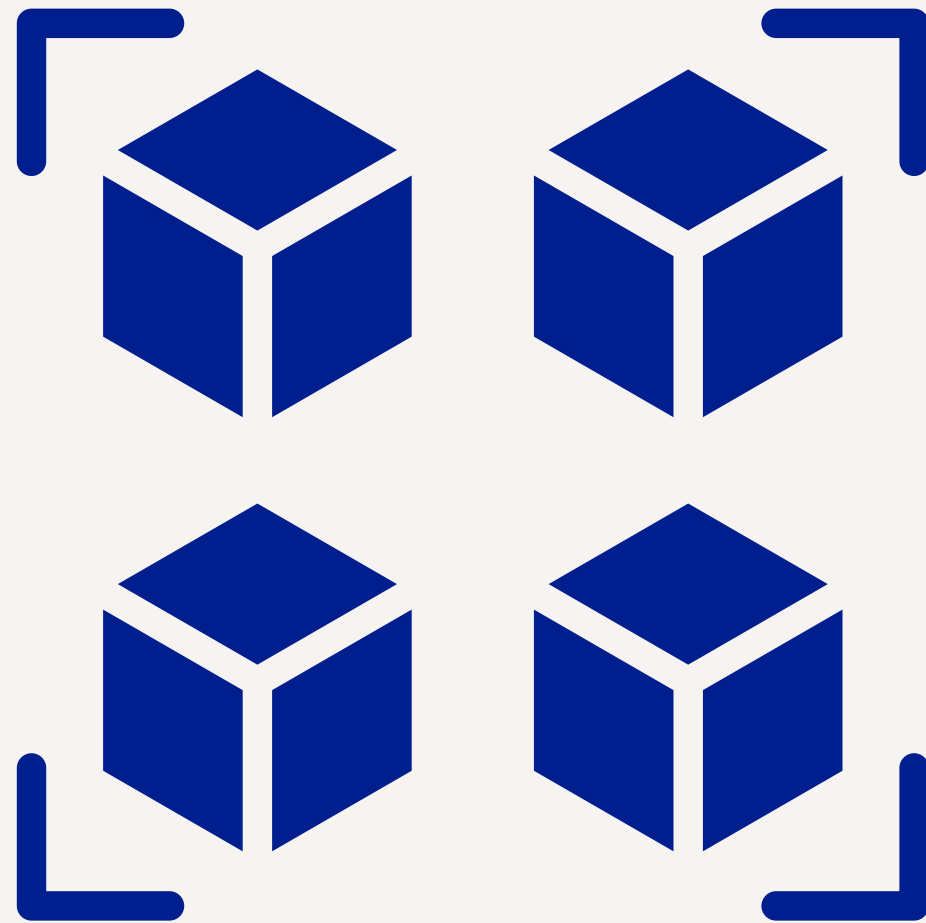
JavaScript

実習8 - アニメーションを加えよう！ - アニメーションの基本的な記述方法

今回の内容 - 教科書準拠

- 1.6-1 動きがあるWEBサイトを見てみよう(スキップ)
- 2.6-2 なぜ動きがあると効果的なのか
- 3.6-3 心地よいと感じる動きとは
- 4.6-4 見出しを下から浮き上がらせよう 【実習】
- 5.6-5 アニメーションの基本の書き方 【実習+】
- 6.6-6 複数のアニメーションを加えよう 【実習+】
- 7.6-7 動きの詳細を加えよう 【実習+】

アニメーションを加えよう！



Chapter6のポイント

1. 複数のアニメーションを使えるようになる
2. 効率的なスクロール管理「Intersection Observer」を理解する

アニメーションとイベント



アニメーション

時間的な変化を与え自然さやダイナミックさを表現することが可能。
∴より自然な反応を見せることが可能。

イベント

あるきっかけを元に別の行動などを起こすこと。

適切なアニメーションとは

適切なアニメーションを構成する要素

- タイミングは適切か
- 速度は早すぎず遅すぎないか
- 動かす数は適切か
- ループ回数は問題ないか
- 自然な動きになっているか
- デザインや内容から浮いていないか

総じて、ユーザーの邪魔、ストレスになっていないか。

少しでもプラスになっていることが大切。

参考例) Amazon「表示が0.1秒遅れると、売上が1%減少する」 ※アニメーションではないが

6-4 見出しを下から浮き上がらせよう【実習】

シンプルなアニメーション実習

アニメーションで指定する内容

- アニメーションをするプロパティ(キーフレーム)
- 再生時間(実行時間)などの設定値。再生時間のみ必須。詳細は6-7で解説。

6-4の実習で扱うレベルのアニメーションに関してはcssでも十分に実装できる。
JSでよく使うシンプルアニメーションの例は「スムーズスクロール」など。

6-5 アニメーションの基本の書き方【実習+】

WebAnimationsAPI

JavaScriptのDOMにアニメーションを適用できる

- cssとは異なりキーフレームはローワーキャメルケースで記述する。頭は小文字で単語の区切りは大文字(例: lowerCamelCase)。類似のものにアッパーキャメルケース(例: UpperCamelCase)。
 - fontSize/paddingTop/scrollTopなど。
 - →形式的にはcssの「-」を消して次の文字を大文字にする。
- 基本形式は、「何を」「どのくらいの時間をかけて」実行するか
 - Element.animate(ターゲット, 再生時間 or オプション)

P Display: none → blockなどは数値で示せないため、アニメーションとして正常に動作しない。

✓ Opacityで解決しよう。イベント発生条件に違いがある点に注意。

6-7 動きの詳細を加えよう【実習+】

オプションを設定しよう

オプションの詳細とよく使う設定値

- 実行時間のみの場合は数値のみで良い
 - 逆に言うと数値のみの場合は必然的に時間となる
 - オーバーロード
- オプション値を設定する場合はオブジェクトで「key: 値」として設定する
 - デフォルト値を用いる場合は指定しなくて良い
- 速度変化の「easing」設定はよく使われる
 - https://www.mitsue.co.jp/knowledge/blog/frontend/201711/29_1427.html

複数のキーフレーム指定

複数のアニメーションを設定する

- 同じオプション値または時間を指定する場合に複数の設定が可能
- 配列形式の指定
 - プロパティの状態を明確にしたい場合
- オブジェクト形式の指定(おすすめ)
 - 開始の状態と終了の状態を明確にしたい場合

今回のまとめ



ネイティブアニメーション

JavaScriptではライブラリを使ったアニメーションがよく使われる。ライブラリの中でネイティブのAPIを使うこともよくあるため、基本的な動きを理解しておこう。



JavaScript

実習9 - アニメーションを加えよう！ - 複雑なアニメーションの記述

今回の内容 - 教科書準拠

- 1.6-8 色々な見出しのアニメーション【実習】
- 2.6-9 複数の画像を順番に表示しよう【実習】
- 3.6-10 すべてのクラスを取得しよう【実習+】
- 4.6-11 1つずつ遅延させよう【実習+】
- 5.6-12 色々な画像のアニメーション【実習】

6-8 色々な見出しのアニメーション【実習】

ダイナミックに動かす

見出しやボタンでよく使われるアニメーションを理解しよう

ボタンの背景色を変更する、ボーダー色を変更するなど、よく使われる手法は覚えておこう。

CSSとJavaScriptのどちらを優先すべきか

CSSの方が効率よく動かすことができるため優先したい。

JavaScriptは他に重要な仕事が多くある。

4-10では「:disabled」を使ったが、同様にJavaScriptでクラスだけ制御し、デザインはCSSで制御する方法もよく使われる。

```
a {  
  color: gray;  
}  
a.is-active {  
  color: blue;  
}
```


6-9 複数の画像を順番に表示しよう【実習】

ダイナミックに動かす

見出しやボタンでよく使われるアニメーションを理解しよう

ボタンの背景色を変更する、ボーダー色を変更するなど、よく使われる手法は覚えておこう。

CSSとJavaScriptのどちらを優先すべきか

CSSの方が効率よく動かすことができるため優先したい。

JavaScriptは他に重要な仕事が多くある。

4-10では「:disabled」を使ったが、同様にJavaScriptでクラスだけ制御し、デザインはCSSで制御する方法もよく使われる。

```
a {  
  color: gray;  
}  
a.is-active {  
  color: blue;  
}
```

アニメーションの組み合わせ

数個のアニメーションでも大きな動きができることを学ぼう

単純な複数のアニメーションでも、教科書の通り意外とダイナミックな動きができる。

✅ アニメーションにつられて操作性を失わないことに注意が必要

今回のまとめ



リアリティのあるアニメーション

わかりやすい画像の操作を通して、それぞれのプロパティが与える役割を学ぶ。ネイティブで作り込む機会が多いとも限らないが、何ができて何が起きているかを理解することは押さえておこう。